Universidad Autónoma de Ciudad Juárez

Instituto de Ingeniería y Tecnología

Departamento de Ingeniería Eléctrica y Computación



Tarea 14 Control de posición de un carrito

Sensores e Instrumentación

Ángel Alvídrez 225581:

Fecha: 30 de Abril 2025

Índice

1. Objetivo **2**

2. Introducción **2**

3. Definiciones y marco conceptual **3**

3.1. Sensor ultrasónico 3

3.2. Control PID 4

3.3. Comunicación Bluetooth 4

3.4. Motores de Corriente Directa (CD) 4

3.5. Señales PWM 5

3.6. Puente H (L298N) 5

4. Desarrollo **6**

5. Código **7**

6. Resultados **10**

7. Conclusión **11**

1. Objetivo

Desarrollar un sistema de control de posicion para un carrito motorizado, utilizando un sensor de ultrasonido y un controlador PID implementado en Arduino, con el fin de mantener una distancia fija respecto a una pared.

2. Introducción

En el ambito de la mecatronica y el control automatico, uno de los retos fundamentales es lograr que un sistema se mantenga en una posicion o trayectoria deseada ante posibles perturbaciones externas. En esta practica se implementa un sistema de control de posicion para un carrito con motores de corriente directa, cuyo objetivo es mantener una distancia constante respecto a una pared utilizando un sensor ultrasónico HC-SR04.

El sistema se controla mediante un algoritmo PID (Proporcional, Integral, Derivativo), el cual ajusta dinamicamente la velocidad y direccion del carrito para minimizar el error entre la distancia actual y la deseada. Esta ultima se establece de forma remota a traves de una interfaz Bluetooth, permitiendo experimentar en tiempo real con distintos valores de referencia.

El uso de control PID en sistemas de posicion es ampliamente utilizado en la industria debido a su simplicidad y efectividad. Esta practica permite al estudiante comprender de manera practica el funcionamiento de un controlador en lazo cerrado, asi como la importancia del ajuste adecuado de sus parametros para obtener una respuesta estable y precisa.

3. Definiciones y marco conceptual

3.1. Sensor ultrasónico

Este sensor permite medir distancias a traves del tiempo de vuelo de un pulso ultrasónico. Al enviar un pulso sonoro de alta frecuencia y detectar el eco reflejado en un objeto, es posible calcular la distancia mediante la formula:

$$\text{distancia} = \frac{340 \text{ m/s} \cdot t(\mu s)}{2} \cdot \frac{1 \text{ s}}{1{,}000{,}000 \ \mu s} \cdot \frac{100 \text{ cm}}{1 \text{ m}}$$

La cual simplificada queda como

$$\text{distancia (cm)} = t(\mu s) \times 0{,}01715$$



Figura 1: Sensor Ultrasónico HC-SR04

3.2. Control PID

El controlador PID (Proporcional, Integral, Derivativo) es uno de los algoritmos más utilizados en el control de sistemas. Su salida se basa en tres componentes:

- Proporcional (P): responde de manera proporcional al error actual.
- Integral (I): acumula el error en el tiempo para corregir errores persistentes.
- Derivativo (D): anticipa la tendencia del error para reducir oscilaciones.

La combinación de estas tres acciones permite lograr un equilibrio entre rapidez, estabilidad y precisión. En este caso, el PID se implementa con la librería PID v1 de ARDUINO en el ESP 32, configurando los parámetros de manera empírica hasta lograr un comportamiento deseado.

3.3. Comunicación Bluetooth

La comunicacion Bluetooth permite enviar y recibir datos de forma inalambrica. En esta practica, se utiliza el modulo integrado del ESP32 junto con la libreria BluetoothSerial para recibir el valor del setpoint en tiempo real desde un dispositivo externo, permitiendo modificar la referencia de posicion sin necesidad de reprogramar el sistema utilizando una aplicación Android creada en MIT App inventor.

3.4. Motores de Corriente Directa (CD)

Los motores de corriente directa convierten energía eléctrica en movimiento rotatorio. Son ampliamente utilizados por su facilidad de control y bajo costo. En esta práctica se utilizaron motores de 6V, los cuales permiten manejar velocidades aceptables para un entorno de laboratorio o salón.

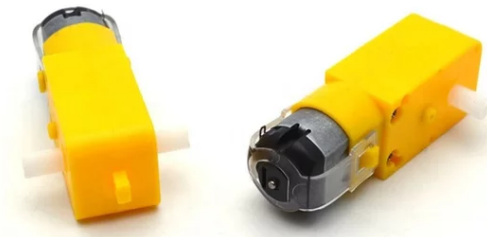


Figura 2: Motor de 6V DC

3.5. Señales PWM

La modulación por ancho de pulso (PWM, por sus siglas en inglés) es una técnica que permite controlar la velocidad de un motor de CD ajustando el ciclo de trabajo (duty cycle) de una señal digital. Cuanto mayor sea el ciclo de trabajo, mayor será la energía suministrada al motor, y por ende, su velocidad.

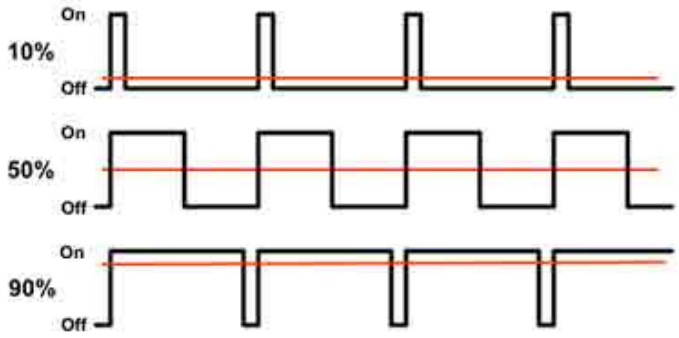


Figura 3: Gráfica de PWM

3.6. Puente H (L298N)

El puente H es un circuito que permite cambiar la polaridad de alimentación de los motores, lo cual permite invertir su sentido de giro. El L298N es un puente doble que

permite controlar dos motores de manera independiente, y también puede controlar su velocidad mediante la señal PWM enviada desde un microcontrolador como el Arduino.

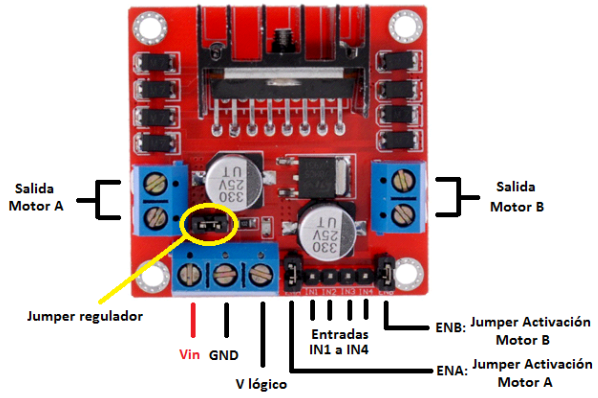


Figura 4: Puente H L298N

4. Desarrollo

El sistema de control de posición se implementó en una placa ESP32, la cual se encargó tanto de la lectura del sensor ultrasónico HC-SR04 como del control de los motores a través del puente H L298N. La programación se realizó en el entorno de Arduino para controlar un ESP32, utilizando la librería PID_v1.h para implementar el controlador PID y la librería `BluetoothSerial.h` para recibir datos desde un dispositivo móvil o computadora mediante Bluetooth.
El flujo del programa es el siguiente:

- Inicialización de la comunicación serial y Bluetooth.
- Configuración de los pines para el control de los motores y el sensor ultrasónico.
- Espera de un valor de setpoint enviado por Bluetooth, el cual representa la distancia deseada en centímetros entre el carrito y la pared.
- Lectura constante de la distancia utilizando el tiempo de vuelo del sensor ultrasónico.
- Cálculo de la señal de control usando el algoritmo PID.

- Aplicación de la señal de control a los motores, utilizando PWM para ajustar la velocidad según la magnitud del error.
- Si el PWM calculado es muy bajo (pero no cero), se impone un valor mínimo para asegurar que los motores reaccionen.

El código se fue ajustando empíricamente, principalmente los valores de las constantes del PID y el umbral mínimo de PWM necesario para que el carrito se mueva, pues con valores muy bajos el carrito no se movía . Se observó que, con las constantes $K_p = 1{,}0$, $K_i = 0{,}0$ y $K_d = 0{,}2$, el comportamiento del sistema era suficientemente estable y respondía de forma adecuada a las correcciones de posición.

5. Código

Este fue el código utilizado explicado anteriormente:

Listing 1: Código de control PID para el carrito

```
#include <PID_v1.h>
#include "BluetoothSerial.h"
BluetoothSerial SerialBT;

// Pines motores L298N
const int M1_A = 16, M1_B = 17;
const int M2_A = 18, M2_B = 19;

// Pines HC-SR04
const int trigPin = 27, echoPin = 26;
long duration;
double leerDistancia() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);  // Opcional, pero puede ayudar a estabilizar
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long duration = pulseIn(echoPin, HIGH);  // sin timeout
  return duration * 0.01715;
```

```
}

// PID
double Setpoint, Input, Output;
double Kp = 1.0, Ki = 0.0, Kd = 0.2;
PID myPID(&Input, &Output, &Setpoint, Kp, Ki, Kd, DIRECT);

// Umbral m nimo de motor
const int motorMinPWM = 135;

void setup() {
  Serial.begin(115200);
  while (!Serial) delay(10);
  Serial.println("\n--- Control PID de distancia ---");

  SerialBT.begin("ESP32_Carrito"); // Nombre Bluetooth

  // Pines motores
  pinMode(M1_A, OUTPUT); pinMode(M1_B, OUTPUT);
  pinMode(M2_A, OUTPUT); pinMode(M2_B, OUTPUT);
  // Pines ultrasonido
  pinMode(trigPin, OUTPUT); pinMode(echoPin, INPUT);

  // Esperar primer Setpoint por Bluetooth
  Serial.println("Esperando Setpoint por Bluetooth (1 byte en cm)...");
  while (SerialBT.available() == 0) delay(10);
  Setpoint = SerialBT.read();
  Serial.print("Setpoint inicial recibido: ");
  Serial.print(Setpoint);
  Serial.println(" cm");

  // Init PID
  Input = leerDistancia();
  myPID.SetMode(AUTOMATIC);
  myPID.SetOutputLimits(-255, 255);

  Serial.println("Setup completo. Iniciando control...");
```

```
}

void loop() {
  // Actualizar Setpoint si llega uno nuevo
  if (SerialBT.available() > 0) {
    Setpoint = SerialBT.read();
    Serial.print("Nuevo Setpoint recibido: ");
    Serial.print(Setpoint);
    Serial.println(" cm");
  }

  // 1) Leer distancia
  Input = leerDistancia();
  Serial.print("Dist actual: "); Serial.print(Input); Serial.println(" cm");

  // 2) Calcular PID
  myPID.Compute();

  // 3) Control de motores
  int pwm = abs((int)Output);
  bool adelante = (Output > 0);

  if (pwm > 0 && pwm < motorMinPWM) pwm = motorMinPWM; // Aplicar m nimo PWM

  if (adelante) {
    analogWrite(M1_A, pwm); analogWrite(M2_A, pwm);
    analogWrite(M1_B, 0);   analogWrite(M2_B, 0);
  } else {
    analogWrite(M1_B, pwm); analogWrite(M2_B, pwm);
    analogWrite(M1_A, 0);   analogWrite(M2_A, 0);
  }

  // 4) Debug en monitor serial
  Serial.print(" SP:"); Serial.print(Setpoint);
  Serial.print("  PV:"); Serial.print(Input);
  Serial.print("  Err:"); Serial.print(Setpoint - Input);
  Serial.print("  Out:"); Serial.print(Output);
```

```
  Serial.print("  PWM:"); Serial.println(pwm);

  delay(100);
}
```

6. Resultados

Durante las pruebas del sistema se utilizó un rango de distancias deseadas entre 10 cm y 50 cm, enviado dinámicamente mediante Bluetooth. En este rango, el carrito mantuvo una posición estable frente a la pared, corrigiendo su distancia de forma continua mediante el controlador PID.

Se observó que, al aumentar la distancia deseada más allá de los 50 o 60 cm, el sensor ultrasónico comenzaba a presentar errores de medición. Esto probablemente se debe a que el sensor opera con un ángulo de detección de aproximadamente 30°, por lo que a distancias grandes puede comenzar a detectar el piso en lugar de la pared, afectando la precisión de la lectura.

Dentro del rango óptimo (10–50 cm), el comportamiento del sistema fue suave, sin oscilaciones muy marcadas ni movimientos muy bruscos. También se comprobó que el uso de un umbral mínimo de PWM (`motorMinPWM`) fue necesario para evitar que los motores quedaran estáticos con señales muy pequeñas, especialmente cuando el error era bajo pero aún necesitaba corrección.

La señal de salida del PID se ajustaba bien a los cambios de setpoint, lo que permitía al carrito adaptarse a nuevas posiciones con una transición rápida y estable. Todo el comportamiento del sistema se podía monitorear en tiempo real desde el monitor serial.

7. Conclusión

El sistema implementado logró controlar efectivamente la posición de un carrito respecto a una pared utilizando un sensor ultrasónico y un controlador PID. La integración del módulo Bluetooth permitió modificar dinámicamente el setpoint, lo cual facilitó la experimentación con distintas distancias objetivo sin necesidad de modificar el código o reiniciar el sistema.
Se comprobó que el sensor ultrasónico es confiable en un rango aproximado de 10 a 50 cm, pero que comienza a fallar a distancias mayores debido a su forma de detección cónica, que puede provocar lecturas erróneas al captar ecos del piso u otros objetos. Esto resalta la importancia de conocer las limitaciones físicas de los sensores al momento de diseñar un sistema de control.
El uso del controlador PID permitió suavizar el comportamiento del carrito, ajustando su posición de forma continua sin necesidad de detenerse por completo. Ajustes empíricos a las constantes K_p, K_i y K_d fueron clave para lograr un desempeño estable y responsivo.
Finalmente, esta práctica fue útil para reforzar conceptos de control automático, electrónica y programación aplicada, mostrando cómo integrar múltiples componentes en un sistema funcional.